

Grado en Informática Industrial y Robotica

Infraestructura Informática para Centros de Procesamiento de Datos

Código de la asignatura: 14541

Curso: 2026-2027

Guion de ejecución

“Creación de máquinas virtuales en Proxmox VE”

Estudiante: Carles Luis Boquera

Profesor responsable: Dr. Lenin G. Lemus Zúniga

Contenido

1. Presentación y alcance del guion 3
2. Objetivos y baremo 4
3. Objetivos 4
4. Baremo de evaluación 4
5. Diseño de la infraestructura 5
6. Máquinas virtuales y direccionamiento 5
7. Topología 6
8. Procedimiento de ejecución 7
9. Paso 1. Creación de las máquinas virtuales en Proxmox 7
10. Paso 2. Instalación del sistema operativo 7
11. Paso 3. Configuración base del sistema 7
12. Paso 4. Servidor web Apache en VM1 y VM2 8
13. Paso 5. Sistema gestor de bases de datos 8
14. Paso 6. Aplicación Flask 9
15. Paso 7. Servicio systemd y proxy inverso con nginx 10
16. Validación 12
17. Validación de los tres servicios 12
18. Evidencias que deben recogerse para la memoria 13
19. Prompts de apoyo con IA 14
20. Lista de verificación final 15

Tabla de Figuras

1. Topología de la infraestructura de la Práctica 4. 6

Listados de código

1. Configuracion basica del sistema operativo 7
2. Instalacion y configuracion del servidor web Apache 8
3. Instalacion y configuracion de PostgreSQL 8
4. Estructura de la aplicacion Flask 9

5. Servicio systemd y proxy inverso 10
6. Comprobacion de los servicios desde otra maquina 12
7. Prompt de consulta sobre la creacion de maquinas virtuales 14
8. Prompt de consulta sobre el documento de diseno 14

1 Presentación y alcance del guion

Este guion desarrolla el procedimiento de ejecución de la **Práctica 4**: la creación y configuración de **tres máquinas virtuales** en Proxmox VE que conforman la infraestructura básica de una red corporativa, basada en **dos servidores web** y **una aplicación Flask** conectada a un sistema gestor de bases de datos.

Obligación previa del estudiante

«Cada alumno debe informar al profesor las características de la aplicación Flask a desarrollar y de los segmentos de red que va a utilizar. Si no la consensua tendrá una penalización de 5 puntos sobre la nota final de la práctica.» Es imprescindible entregar este documento **antes** de la sesión de laboratorio.

2 Objetivos y baremo

3 Objetivos

- Crear y configurar máquinas virtuales en Proxmox VE.
- Interconectar las máquinas virtuales mediante un puente de red (*bridge*) de Proxmox.
- Instalar y configurar un servidor web Apache en dos máquinas virtuales.
- Diseñar e implementar una aplicación web con Flask que acceda a un SGBD.

4 Baremo de evaluación

Tabla 1. Criterios de evaluación de la Práctica 4 y su peso.

Criterio	**Descripción**	**Peso**
Primer servidor web	Funciona el primer servidor web.	10 %
Ambos servidores web	Funcionan los dos servidores web.	10 %
Aplicación Flask	Funciona Flask.	30 %
Integración completa	Los servicios web y la aplicación Flask funcionan correctamente.	50 %

5 Diseño de la infraestructura

6 Máquinas virtuales y direccionamiento

Tabla 2. Máquinas virtuales de la Práctica 4 y su direccionamiento.

ID	Nombre	Dirección IP	Puerta de enlace	Servicio
VM1	`web1`	192.168.100.11	192.168.100.1	Servidor web Apache con la página de la asignatura, el nombre del estudiante y su IP.
VM2	`web2`	192.168.100.12	192.168.100.1	Segundo servidor web Apache, con la misma estructura de página.
VM3	`app`	192.168.100.13	192.168.100.1	Aplicación Flask con acceso al SGBD y servidor web de acceso.
SGBD	`db`	192.168.100.14	192.168.100.1	PostgreSQL, accesible únicamente desde `app`.

Direcciones MAC reservadas

El laboratorio reserva las direcciones MAC de las máquinas virtuales por equipo y grupo:

- Grupo con horario 17:00–18:30: direcciones `mv1XX`, donde `XX` es el número de ordenador del laboratorio.
- Grupo con horario 18:30–20:00: direcciones `mv2XX`.

Las MAC deben fijarse en la configuración de la tarjeta de red de cada máquina virtual, dentro de Proxmox, antes de arrancarla.

7 Topología

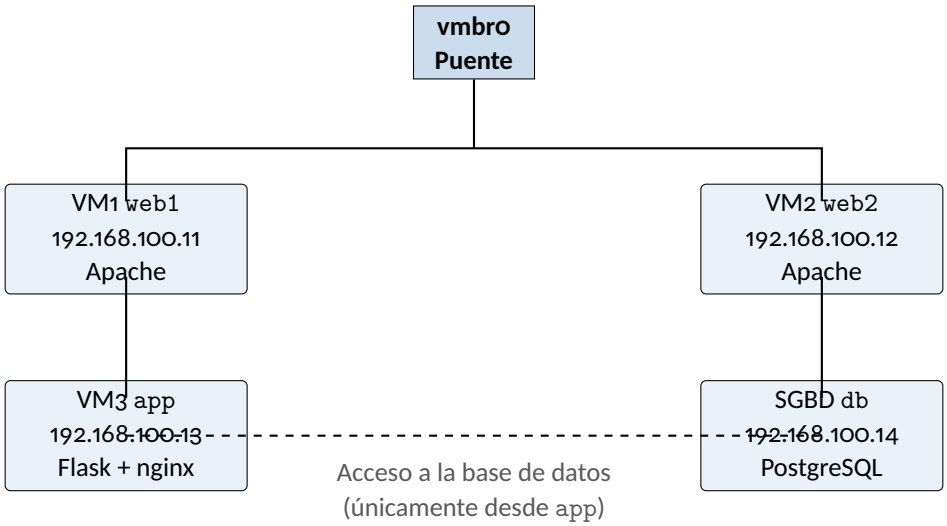


Figura 1. Topología de la infraestructura de la Práctica 4.

8 Procedimiento de ejecución

9 Paso 1. Creación de las máquinas virtuales en Proxmox

1. Acceder a la interfaz web de Proxmox en `https://<host>:8006` con las credenciales del laboratorio.
2. Crear la **máquina virtual** desde `local` → `Create VM`:
   - `General`: identificador `101`, nombre `web1`.
   - `OS`: ISO de Debian 12 (*netinst*). Descargar la ISO desde `https://www.debian.org/downloads/`.
   - `System`: 2 vCPU, 2048 MB de RAM, 32 GB de disco (formato `SCSI`).
   - `Disks`: añadir un discoadditional de 8 GB si se va a servir el repositorio de la aplicación.
   - `Network`: `vmbr0`, con la **dirección MAC reservada** del equipo y el modelo `virtio`.
   - `Confirm`: revisar y Finish.
3. Repetir para `web2` (ID 102) y `app` (ID 103).

Instalar la ISO en el almacenamiento de Proxmox

Desde `local` → `ISO Images` → `Upload`, se descarga la ISO de Debian al almacenamiento de Proxmox. En el laboratorio probablemente ya esté disponible.

10 Paso 2. Instalación del sistema operativo

1. Arrancar la máquina virtual y abrir la consola (`Console`).
2. Seleccionar el idioma, el país y el teclado.
3. En *Network configuration*, configurar la interfaz con la dirección IP estática de la tabla 2, la máscara `255.255.255.0` y la puerta de enlace `192.168.100.1`.
4. Marcar `SSH server` y `web server` en la selección de paquetes.
5. Completar la instalación y reiniciar.

11 Paso 3. Configuración base del sistema

```
# Actualizar el sistema
apt update && apt upgrade -y

# Instalar utilidades de red y de diagnostico
apt install -y curl wget net-tools iproute2 dnsutils bridge-utils

# Fijar la direccion IP de forma permanente
cat > /etc/network/interfaces.d/red << 'EOF'
auto ens18
iface ens18 inet static
    address 192.168.100.11
    netmask 255.255.255.0
    gateway 192.168.100.1
    dns-nameservers 192.168.100.1
EOF
```

```

# Comprobar la conectividad
ip a
ping -c 3 192.168.100.1
```

Listado 1. Configuracion basica del sistema operativo

12 Paso 4. Servidor web Apache en VM1 y VM2

```
# Instalar Apache
apt install -y apache2

# Publicar la pagina de la asignatura, el nombre del estudiante y su IP
cat > /var/www/html/index.html << 'EOF'
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>ICD - Infraestructura Informatica para Centros de Datos
    </title></head>
<body>
  <h1>Infraestructura Informatica para Centros de Datos</h1>
  <p>Estudiante: Carles Luis Boquera</p>
  <p>Direccion IP de este servidor: 192.168.100.11</p>
</body>
</html>
EOF

# Ajustar el documento de inicio por defecto de Apache
sed -i 's#/var/www/html/index.html#/var/www/html/index.html#g' /etc/apache2/sites-
    available/000-default.conf
systemctl enable apache2 && systemctl restart apache2
```

Listado 2. Instalacion y configuracion del servidor web Apache

13 Paso 5. Sistema gestor de bases de datos

El SGBD se instala en la máquina app (o en una máquina adicional si el profesor lo autoriza). En este guion se usa PostgreSQL.

```
apt install -y postgresql postgresql-contrib
systemctl enable postgresql && systemctl restart postgresql

# Crear el rol y la base de datos de la aplicacion
sudo -u postgres psql << 'EOF'
CREATE USER icdapp WITH PASSWORD 'icdapp2026';
CREATE DATABASE icdapp OWNER icdapp;
GRANT ALL PRIVILEGES ON DATABASE icdapp TO icdapp;
EOF

# Permitir el acceso desde la red
sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '*'/" /etc/postgresql/16/main
    /postgresql.conf
sed -i "s/#port = 5432/port = 5432/" /etc/postgresql/16/main/postgresql.conf
# Restringir el acceso a la red de la practica en pg_hba.conf
echo "host icdapp icdapp 192.168.100.0/24 scram-sha-256" >> /etc/postgresql/16/main/pg_hba.
    conf
systemctl restart postgresql
```

Listado 3. Instalacion y configuracion de PostgreSQL

Principio de mínimo privilegio

La línea añadida a `pg_hba.conf` limita el acceso a la base de datos a la red `192.168.100.0/24`. Es la medida mínima razonable: la base de datos de una aplicación no debe ser accesible desde cualquier origen, y menos aún desde Internet, como se verá en la Práctica 5.

14 Paso 6. Aplicación Flask

```
mkdir -p /opt/icdapp
cd /opt/icdapp
python3 -m venv venv
source venv/bin/activate
pip install flask flask-sqlalchemy psycopg2-binary gunicorn

cat > app.py << 'EOF'
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy
import os, socket

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://icdapp:icdapp2026@127.0.0.1:5432/
    icdapp'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class Estudiante(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nombre = db.Column(db.String(80), nullable=False)
    asignatura = db.Column(db.String(80), nullable=False)
    ip = db.Column(db.String(45), nullable=False)

@app.route('/')
def index():
    filas = Estudiante.query.all()
    return render_template('index.html', filas=filas, ip=socket.gethostbyname(socket.
    gethostname()))

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
        if Estudiante.query.count() == 0:
            db.session.add(Estudiante(nombre='Carles Luis Boquera',
                                      asignatura='Infraestructura Informatica para
    Centros de Datos',
                                      ip=socket.gethostbyname(socket.gethostname())))
            db.session.commit()
    app.run(host='0.0.0.0', port=5000)
EOF

cat > templates/index.html << 'EOF'
<!DOCTYPE html>
<html lang="es"><head><meta charset="utf-8"><title>ICD - Aplicacion Flask</title></head>
```

```
<body>
  <h1>Infraestructura Informatica para Centros de Datos</h1>
  <p>Estudiante: Carles Luis Boquera</p>
  <p>Direccion IP: {{ ip }}</p>
  <table border="1">
    <tr><th>Nombre</th><th>Asignatura</th><th>IP</th></tr>
    {% for f in filas %}<tr><td>{{ f.nombre }}</td><td>{{ f.asignatura }}</td><td>{{ f.ip }}</td></tr>{% endfor %}
  </table>
</body></html>
EOF
```

Listado 4. Estructura de la aplicacion Flask

15 Paso 7. Servicio systemd y proxy inverso con nginx

```
cat > /etc/systemd/system/icdapp.service << 'EOF'
[Unit]
Description=Aplicacion Flask de la asignatura ICD
After=network.target postgresql.service

[Service]
Type=simple
User=root
WorkingDirectory=/opt/icdapp
Environment="PATH=/opt/icdapp/venv/bin"
ExecStart=/opt/icdapp/venv/bin/gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now icdapp

# Proxy inverso con nginx
apt install -y nginx
cat > /etc/nginx/sites-available/icdapp << 'EOF'
server {
    listen 80;
    server_name _;
    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF
ln -sf /etc/nginx/sites-available/icdapp /etc/nginx/sites-enabled/icdapp
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx
```

Listado 5. Servicio systemd y proxy inverso

Por qué nginx y no Flask directamente

Ejecutar el servidor de desarrollo de Flask (`app.run`) en producción es una mala práctica: es de un solo hilo, no gestiona bien las peticiones concurrentes y no termina las conexiones de forma ordenada. `gunicorn` aporta concurrencia y nginx aporta terminación TLS, registro de accesos y cacheo de estáticos en la Práctica 5.

16 Validación

17 Validación de los tres servicios

```
# Servidor web de la VM1
curl -s http://192.168.100.11 | head -5

# Servidor web de la VM2
curl -s http://192.168.100.12 | head -5

# Aplicacion Flask (a traves de nginx)
curl -s http://192.168.100.13 | head -10

# La aplicacion responde y lee de la base de datos
curl -s http://192.168.100.13 | grep -i "Carles Luis Boquera"
```

Listado 6. Comprobacion de los servicios desde otra maquina

Tabla 3. Comprobaciones de la Práctica 4 y su resultado esperado.

Comprobación	**Resultado esperado**
El servicio Apache está activo en VM1 y VM2.	`systemctl status apache2` muestra `active (running)`.
Las páginas web muestran asignatura, nombre e IP.	La respuesta HTTP contiene los tres datos.
El servicio `icdapp` está activo.	`systemctl status icdapp` muestra `active (running)`.
La aplicación Flask responde en el puerto 80.	`curl` devuelve el código de salida `0` y el HTML de la aplicación.
La aplicación lee de la base de datos.	La tabla de la página muestra al menos una fila.
La base de datos no es accesible desde otras máquinas.	`psql` desde otra máquina devuelve un error de autenticación o de acceso.

18 Evidencias que deben recogerse para la memoria

Tabla 4. Evidencias a capturar durante la ejecución.

#	Evidencia	Apartado de la memoria
1	Captura de la creación de la máquina virtual en Proxmox.	Desarrollo: creación
2	Configuración de red de cada VM con su IP y su MAC reservada.	Desarrollo: configuración
3	Página web servida por VM1 y por VM2.	Desarrollo: servidores web
4	Página de la aplicación Flask con los datos de la base de datos.	Desarrollo: aplicación
5	Estado de los servicios `apache2`, `icdapp` y `nginx`.	Validación
6	Salida de `curl` contra los tres servicios.	Validación
7	Fichero `pg_hba.conf` con la restricción de acceso.	Validación: seguridad

19 Prompts de apoyo con IA

```
Estoy realizando la Practica 4 de Infraestructura Informatica para Centros de
Datos. Debo crear tres maquinas virtuales en Proxmox VE (web1, web2 y app) que
conformen una red corporativa con dos servidores web Apache y una aplicacion Flask
con acceso a PostgreSQL.

Proporcame:
1. La secuencia de pasos exacta para crear una maquina virtual en Proxmox VE desde
   la interfaz web, indicando el orden de las pestanas y los parametros que
   deben fijarse (CPU, RAM, disco, red, modo de arranque).
2. Los comandos de Debian para instalar y configurar Apache con una pagina que
   muestre el nombre de la asignatura, el nombre del estudiante y la direccion IP
   del servidor, incluyendo como fijar la IP de forma permanente en Debian 12.
3. Como crear una aplicacion Flask minima con acceso a PostgreSQL mediante
   SQLAlchemy, con una tabla y una consulta que devuelva sus filas, y como
   ejecutarla como servicio systemd con gunicorn y un proxy inverso con nginx.
4. Que medidas de seguridad minimas aplicables a la base de datos para que solo
   sea accesible desde la red de la aplicacion.
No inventes referencias bibliograficas.
```

Listado 7. Prompt de consulta sobre la creacion de maquinas virtuales

```
Voy a informar al profesor del diseno de la Practica 4 antes de la sesion de
laboratorio. Redactame un breve documento (una pagina) con:
1. Un diagrama de la topologia de las tres maquinas virtuales y el SGBD, con los
   segmentos de red, los nombres de cada componente y sus direcciones IP.
2. La caracterizacion de la aplicacion Flask: que datos va a guardar, que esquema
   de base de datos usa, y como se mostraran en la pagina web.
3. Los puertos y servicios que se van a exponer, indicando cuales quedan
   restrictedidos a la red interna.
Redactalo en formato plano, sin markdown, para poder pegarlo en un documento.
```

Listado 8. Prompt de consulta sobre el documento de diseno

20 Lista de verificación final

Tabla 5. Comprobaciones antes de entregar la Práctica 4.

#	Comprobación
1	El documento de diseño (aplicación Flask y segmentos de red) se ha entregado al profesor antes de la sesión.
2	Las tres máquinas virtuales están creadas, con la MAC reservada del equipo.
3	Cada máquina tiene su IP estática y su puerta de enlace correctas.
4	Apache está activo en VM1 y VM2, y las páginas muestran asignatura, nombre e IP.
5	PostgreSQL está activo, con la base de datos y el usuario de la aplicación.
6	La aplicación Flask lee de la base de datos y muestra los datos en la página.
7	El servicio systemd `icdapp` arranca con la máquina y se reinicia ante un fallo.
8	La base de datos no es accesible desde fuera de la red de la práctica.